Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Lambda Function

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Lambda Function
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Lambda Function

1 Lambda Function

Study focus: Lambda Function. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000038 · EV0000039

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In today's class, we are going to be looking into the concept of a lambda.

Key details

- Lambda is basically a function without any name.
- There will not be any name for the functions.
- function without name.
- They're also called as they're famously called as anonymous functions.
- They're called as anonymous functions.
- They are also called as throw away functions.
- Lambda is not a statement, but it's an expression.
- And one important thing that you should always know with respect to lambda is lambda's body does not contain a block of statement.
- You will not have a block of statement.
- Instead, it's going to be a single expression.
- It's a single expression.
- I will basically use a lambda function when I do not want to use a function again and again.
- But in this particular case, if I make use of lambda, which means that I do not want to use this function again and again.
- And I will also make use of a lambda function whenever I need a function object.
- And a function object is required.
- the today when I use a lambda function, you may not appreciate the usage of the lambda functions.
- You want to use it for a shorter duration.
- The syntax for the lambda function is you will have lambda, it's a keyword.

- Lambda is a keyword which is used to define or declare an anonymous function.
- And I'll say either it you can have a single argument or you can have a number of arguments.
- This is fine.
- They all needs to be separated by a comma.
- Followed by a colon.
- A colon is a must.
- And then it's an expression.
- You will have an expression.
- And the results are returned back.
- Whenever you're working with a lambda function, you should never return a value.
- Do not return anything because it has an implicit written.
- If you cannot write a function using a lambda, then the regular way of writing you should always make use.

Example or application from the lecture

- Let's try to understand what exactly is a lambda, then we will also learn the usage of lambda.
- For example, in the previous sessions you have already seen how to create a function, and we have also given a name for a function, which means that I can reuse a function as and when it.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000038 · EV0000039 · EV0000040 · +1 source units in pack.json

Core explanation · 05:00–09:59

If your intention is to work with an inline function,.

Key details

- Fine, a single line of code, then always better to use a lambda function.
- X, Y, and Z is what I will try to write.
- And then I will say return.
- X plus Y plus Z.
- A simple one.
- X plus Y plus Z.
- This is what I'm trying to do.
- I will call the function.
- Two, three, and four.
- I'll say print key.
- I know it is the same program that you have already done.
- What we will try to do is we will try to convert this function into a lambda function.
- Let me convert this into a lambda function.
- A lambda as a keyword.
- You can have a single.

- If you have a single argument, you will specify it like this, then followed by a colon.
- And then followed by your expression.
- But in this case, if you can see that there are how many number of arguments are there?
- There are basically three arguments are there.
- And what are we trying to do?
- We are trying to perform addition in this case.
- This is what I will try to do.
- Two, three, comma four.
- This is how you try to write a lambda function.
- F is basically getting a function object.
- This function object is when a lambda expression creates it.
- Define also will work in a similar fashion, okay?
- DEF when we use it, it works in a similar fashion.
- But our assignment is going to happen automatically.
- In this case, the function object is available only when a lambda expression is created.
- Let me go here execute both of them.
- Hope the concept of lambda is clear to you.
- I'll write a program and I want you people to find out what I'm trying to do.
- And an argument is gonna be a single argument.
- I'll say A star A star A.
- I'm trying to find a cube of a particular number.
- That's it.
- Nothing else I will have to do.
- X of five.
- And you may also specify as it is a print statement Q value is Okay, so this is what something I will do.
- Function object X.
- X is a function object in this case.
- Sorry, let me execute this program.
- And C nine over the previous programs output because it's not been commented, and then followed by the Q value of five is what we were trying to find out.
- I request you to solve some of the other examples in order to understand the concept of lambda in a better way.

Example or application from the lecture

- And maybe we'll take up one more simple example and then we'll try to write.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000042 · EV0000043 · EV0000044 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Lambda Function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lambda Function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Lambda Function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9992; only the JSON segmentation supplies the displayed timestamps.
Lambda Function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/008_Lambda_Function__4c5d893e819615b4.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.

4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00014	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...
SRC00015	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00016	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...
SRC00017	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare...

C Revision and retrieval practice